## Question 1 – Fetch API CRUD Practice Page

Q1.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fetch API CRUD Practice</title>
  <link rel="stylesheet" href="Q1_crud.css">
  <script src="Q1_crud.js" defer></script>
</head>
<body>
  <header class="page-header">
    <h1>Fetch API CRUD Practice</h1>
    <p class="methods">GET <span>•</span> POST <span>•</span> PUT <span>•</span> PATCH
<span>•</span> DELETE</p>
  </header>

  <main class="container">
    <section class="panel">
      <h2>Add Post</h2>
      <form id="postForm">
        <label for="postTitle">Post title</label>
        <input id="postTitle" name="title" type="text" placeholder="Enter post title" required>
        <label for="userId">User ID</label>
        <input id="userId" name="userId" type="number" min="1" value="1" required>
        <label for="postBody">Post content</label>
        <textarea id="postBody" name="body" rows="4" placeholder="Write your post..."
required></textarea>
        <div class="button-row">
          <button type="submit">Add Post (POST)</button>
          <button type="button" id="reloadButton" class="secondary">Reload Posts (GET)</button>
        </div>
      </form>
      <p id="message" class="message" role="status">Ready.</p>
      <p id="statusLog" class="status-log">HTTP status: —</p>
    </section>
    <section>
      <div class="section-heading">
        <h2>Posts</h2>
        <span id="postCount">0 posts</span>
      </div>
      <div id="postsContainer" class="posts-grid">
        <p>Loading posts...</p>
      </div>
    </section>
  </main>
</body>
</html>
```

Q1.css

```
* { box-sizing: border-box; }
body {
  margin: 0; font-family: Arial, Helvetica, sans-serif; color: #202938;
  background: #f3f6fb; line-height: 1.5;
}
.page-header { background: #183b67; color: white; padding: 28px 18px; text-align: center; }
.page-header h1 { margin: 0 0 8px; font-size: clamp(1.6rem, 4vw, 2.3rem); }
```

```
.methods { margin: 0; color: #d9e8ff; letter-spacing: 1px; font-weight: bold; }
.methods span { padding: 0 7px; color: #7ed8cf; }
.container { width: min(1100px, 92%); margin: 28px auto; }
.panel { background: white; border-radius: 12px; padding: 22px; box-shadow: 0 4px 18px #172b4d12;
margin-bottom: 28px; }
h2 { margin-top: 0; }
label { display: block; margin: 12px 0 5px; font-weight: bold; font-size: .94rem; }
input, textarea { width: 100%; padding: 11px 12px; border: 1px solid #cbd5e1; border-radius: 7px; font
inherit; background: #fff; }
input:focus, textarea:focus { outline: 2px solid #93c5fd; border-color: #2563eb; }
textarea { resize: vertical; }
.button-row { display: flex; flex-wrap: wrap; gap: 10px; margin-top: 16px; }
button { border: 0; border-radius: 7px; padding: 10px 14px; color: white; background: #2563eb; cursor:
pointer; font-weight: bold; }
button:hover { filter: brightness(.93); }
button:disabled { opacity: .55; cursor: wait; }
button.secondary { background: #475569; }
button.danger { background: #b91c1c; }
button.green { background: #047857; }
button.orange { background: #b45309; }
.message { min-height: 1.5em; margin-bottom: 0; font-weight: bold; }
.message.error { color: #b91c1c; }
.message.success { color: #047857; }
.status-log { color: #475569; font-size: .9rem; margin: 4px 0 0; }
.section-heading { display: flex; justify-content: space-between; align-items: center; gap: 10px; }
.section-heading span { color: #64748b; }
.posts-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(100%, 300px), 1fr));
gap: 16px; }
.post-card { background: white; border: 1px solid #e2e8f0; border-radius: 12px; padding: 18px; box-
shadow: 0 3px 12px #172b4d0b; overflow-wrap: anywhere; }
.post-meta { display: flex; justify-content: space-between; color: #64748b; font-size: .82rem; gap: 8px; }
.post-card h3 { margin: 12px 0 8px; color: #183b67; }
.post-card p { white-space: pre-wrap; }
.card-actions { display: flex; flex-wrap: wrap; gap: 7px; margin-top: 14px; }
.card-actions button { font-size: .78rem; padding: 8px 10px; }
.hint { color: #64748b; font-size: .86rem; }
@media (max-width: 520px) { .panel { padding: 16px; } .button-row button { width: 100%; } }
```

## Q1.js

```
const API_URL = "https://jsonplaceholder.typicode.com/posts";
const postsContainer = document.getElementById("postsContainer");
const postForm = document.getElementById("postForm");
const message = document.getElementById("message");
const statusLog = document.getElementById("statusLog");
const reloadButton = document.getElementById("reloadButton");
const postCount = document.getElementById("postCount");

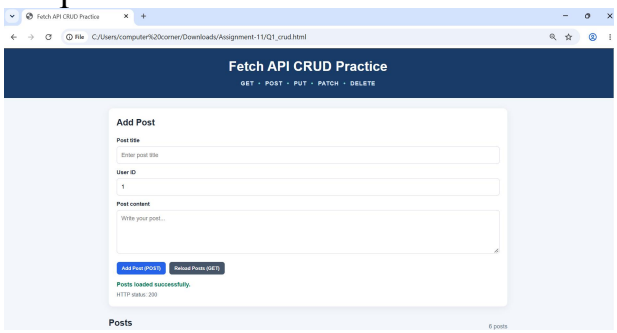
// Keep a local copy because JSONPlaceholder does not permanently save changes.
let posts = [];
function showMessage(text, type = "") {
  message.textContent = text;
  message.className = `message ${type}`;
}
function logStatus(response) {
  statusLog.textContent = `HTTP status: ${response.status} ${response.statusText}`;
  console.log("HTTP status:", response.status, response.statusText);
}
async function loadPosts() {
  postsContainer.innerHTML = "<p>Loading posts...</p>";
```

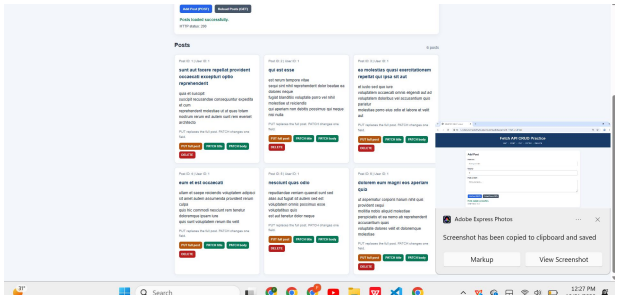
```
  reloadButton.disabled = true;
  showMessage("Loading the first six posts...");
  try {
    const response = await fetch(`${API_URL}?_limit=6`);
    logStatus(response);
    if (!response.ok) throw new Error(`Could not load posts (HTTP ${response.status}).`);
    posts = await response.json();
    renderPosts();
    showMessage("Posts loaded successfully.", "success");
  } catch (error) {
    postsContainer.innerHTML = "";
    showMessage(error.message || "Network error while loading posts.", "error");
  } finally {
    reloadButton.disabled = false;
  }
}
function renderPosts() {
  postCount.textContent = `${posts.length} post${posts.length === 1 ? "" : "s"}`;
  postsContainer.innerHTML = "";
  if (posts.length === 0) {
    postsContainer.innerHTML = "<p>No posts to display.</p>";
    return;
  }
  posts.forEach((post) => {
    const card = document.createElement("article");
    card.className = "post-card";
    const meta = document.createElement("div");
    meta.className = "post-meta";
    meta.textContent = `Post ID: ${post.id} | User ID: ${post.userId}`;
    const title = document.createElement("h3");
    title.textContent = post.title;
    const body = document.createElement("p");
    body.textContent = post.body;
    const hint = document.createElement("p");
    hint.className = "hint";
    hint.textContent = "PUT replaces the full post. PATCH changes one field.";
    const actions = document.createElement("div");
    actions.className = "card-actions";
    const putButton = makeButton("PUT full post", "orange", () => updatePost(post.id, "PUT"));
    const patchTitleButton = makeButton("PATCH title", "green", () => updatePost(post.id, "PATCH",
"title"));
    const patchBodyButton = makeButton("PATCH body", "green", () => updatePost(post.id, "PATCH",
"body"));
    const deleteButton = makeButton("DELETE", "danger", () => deletePost(post.id));
    actions.append(putButton, patchTitleButton, patchBodyButton, deleteButton);
    card.append(meta, title, body, hint, actions);
    postsContainer.appendChild(card);
  });
}
function makeButton(label, colorClass, onClick) {
  const button = document.createElement("button");
  button.type = "button";
  button.textContent = label;
  button.className = colorClass;
  button.addEventListener("click", onClick);
  return button;
}
async function updatePost(id, method, field = "") {
  const existingPost = posts.find((post) => post.id === id);
  if (!existingPost) return;
```

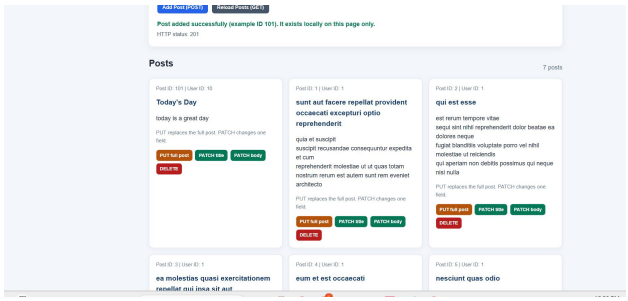
```
  let updatedData;
  if (method === "PUT") {
    updatedData = {
      userId: existingPost.userId,
      title: `${existingPost.title} (updated with PUT)`,
      body: `${existingPost.body} [Full post replaced locally]`
    };
  } else if (field === "title") {
    updatedData = { title: `${existingPost.title} (patched)` };
  } else {
    updatedData = { body: `${existingPost.body} [Body patched]` };
  }
  try {
    showMessage(`Sending ${method} request for post ${id}...`);
    const response = await fetch(`${API_URL}/${id}`, {
      method,
      headers: { "Content-Type": "application/json; charset=UTF-8" },
      body: JSON.stringify(updatedData)
    });
    logStatus(response);
    if (!response.ok) throw new Error(`${method} failed (HTTP ${response.status}).`);
    const returnedPost = await response.json();
    posts = posts.map((post) => post.id === id
      ? (method === "PUT"
          ? { ...post, ...updatedData, ...{ id: post.id } }
          : { ...post, ...updatedData })
      : post);
    renderPosts();
    showMessage(`${method} successful for post ${id}. The page was updated locally; the API does not
permanently save it.`, "success");
    console.log(`${method} response:`, returnedPost);
  } catch (error) {
    showMessage(error.message || `Network error during ${method}.`, "error");
  }
}
async function deletePost(id) {
  try {
    showMessage(`Sending DELETE request for post ${id}...`);
    const response = await fetch(`${API_URL}/${id}`, { method: "DELETE" });
    logStatus(response);
    if (!response.ok) throw new Error(`DELETE failed (HTTP ${response.status}).`);
    await response.json().catch(() => null);
    posts = posts.filter((post) => post.id !== id);
    renderPosts();
    showMessage(`Post ${id} deleted from this page. The API does not permanently save the change.`,
"success");
  } catch (error) {
    showMessage(error.message || "Network error during DELETE.", "error");
  }
}
postForm.addEventListener("submit", async (event) => {
  event.preventDefault();
  const title = document.getElementById("postTitle").value.trim();
  const userId = Number(document.getElementById("userId").value);
  const body = document.getElementById("postBody").value.trim();
  if (!title || !body || !Number.isInteger(userId) || userId < 1) {
    showMessage("Enter a title, content, and valid positive user ID.", "error");
    return;
  }
  try {
```

```
    showMessage("Adding post...");
    const response = await fetch(API_URL, {
      method: "POST",
      headers: { "Content-Type": "application/json; charset=UTF-8" },
      body: JSON.stringify({ title, body, userId })
    });
    logStatus(response);
    if (!response.ok) throw new Error(`POST failed (HTTP ${response.status}).`);
    const newPost = await response.json();
    // Add it to the browser's local state so the new post appears immediately.
    posts.unshift({ ...newPost, title, body, userId });
    renderPosts();
    postForm.reset();
    document.getElementById("userId").value = "1";
    showMessage(`Post added successfully (example ID ${newPost.id}). It exists locally on this page
only.`, "success");
  } catch (error) {
    showMessage(error.message || "Network error while adding the post.", "error");
  }
});
reloadButton.addEventListener("click", loadPosts);
loadPosts();
```

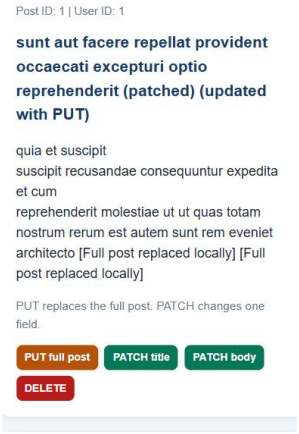
# Output:



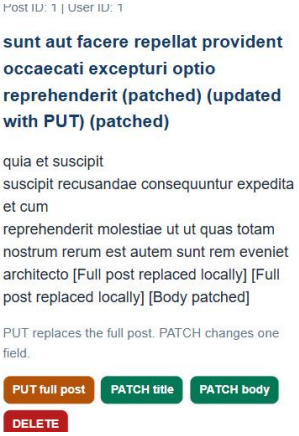



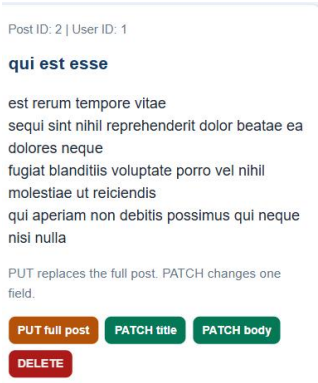
PUT FULL POST




PATCH TITLE




DELETE




# Question 2 – Product Explorer

Q2.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Product Explorer</title>
  <link rel="stylesheet" href="Q2_products.css">
  <script src="Q2_products.js" defer></script>
</head>
<body>
  <header class="hero">
    <p class="eyebrow">ONLINE STORE</p>
    <h1>Product Explorer</h1>
    <p>Explore products and compare their prices.</p>
  </header>
  <main class="container">
    <p id="statusMessage" class="status" role="status">Loading products...</p>
    <p id="httpStatus" class="small">HTTP status: waiting...</p>
    <section id="productsGrid" class="grid" aria-label="Products"></section>
  </main>
</body>
</html>
```

Q2.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, Helvetica, sans-serif; background: #f5f7fb; color: #202938; }
.hero { text-align: center; padding: 42px 16px; color: white; background: linear-gradient(135deg, #253b80, #4776c8); }
.hero h1 { margin: 6px 0; font-size: clamp(2rem, 5vw, 3rem); }
.hero p { margin: 8px 0 0; }
.eyebrow { font-size: .78rem; letter-spacing: 3px; font-weight: bold; color: #dce7ff; }
.container { width: min(1100px, 92%); margin: 26px auto; }
.status { font-weight: bold; }
.small { color: #64748b; font-size: .9rem; }
.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(100%, 230px), 1fr)); gap: 18px; margin-top: 20px; }
.product-card { overflow: hidden; background: white; border: 1px solid #e2e8f0; border-radius: 12px; box-shadow: 0 4px 14px #172b4d0c; }
```

```
.product-image { width: 100%; height: 190px; object-fit: contain; padding: 16px; background:
#fff; }
.product-info { padding: 16px; border-top: 1px solid #edf0f5; }
.product-info h2 { font-size: 1.05rem; margin: 0 0 8px; }
.brand { color: #64748b; margin: 0 0 12px; }
.price { color: #1d4ed8; font-size: 1.25rem; font-weight: bold; margin: 0; }
.error { color: #b91c1c; }
```

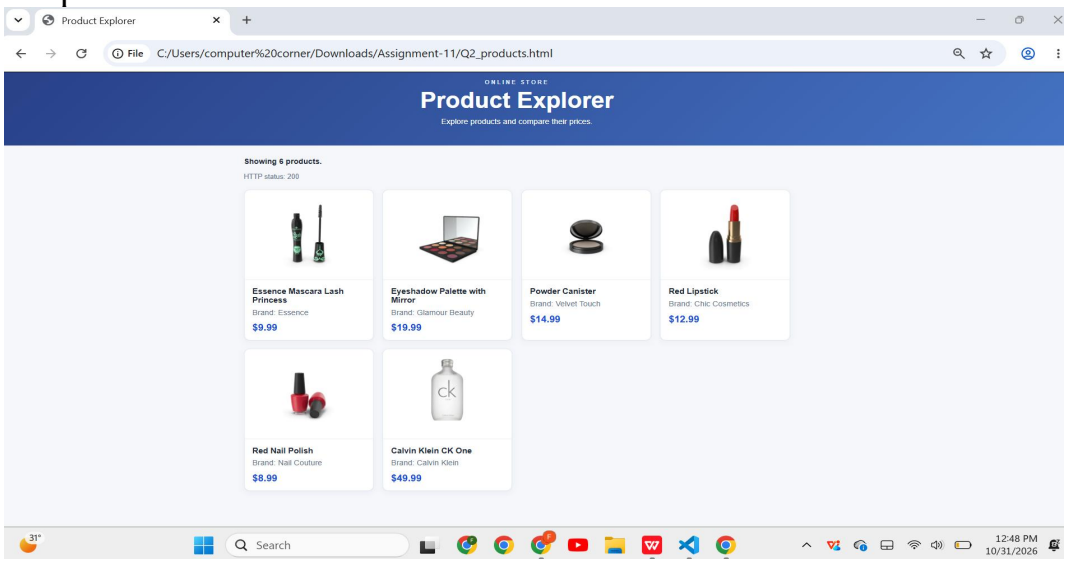
Q2.js

```
const API_URL = "https://dummyjson.com/products";
const productsGrid = document.getElementById("productsGrid");
const statusMessage = document.getElementById("statusMessage");
const httpStatus = document.getElementById("httpStatus");

function showError(message) {
 statusMessage.textContent = message;
 statusMessage.className = "status error";
}
fetch(API_URL)
 .then((response) => {
  // Fetch does not reject automatically for HTTP errors like 404.
  httpStatus.textContent = `HTTP status: ${response.status} ${response.statusText}`;
  console.log("Product API status:", response.status);
  if (!response.ok) {
   throw new Error(`The product request failed (HTTP ${response.status}).`);
  }
  return response.json();
 })
 .then((data) => {
  const products = data.products;
  if (!Array.isArray(products) || products.length === 0) {
   throw new Error("The API did not return any products.");
  }
  console.log("Brand of first product:", products[0].brand || "Brand not provided");
  const firstSixProducts = products.slice(0, 6);
  statusMessage.textContent = `Showing ${firstSixProducts.length} products.`;
  statusMessage.className = "status";
  firstSixProducts.forEach((product) => {
   const card = document.createElement("article");
   card.className = "product-card";
   const image = document.createElement("img");
   image.className = "product-image";
   image.alt = product.title || "Product image";
   image.loading = "lazy";
   image.src = product.thumbnail || "";
   if (!product.thumbnail) image.hidden = true;
   image.addEventListener("error", () => { image.hidden = true; });
   const info = document.createElement("div");
   info.className = "product-info";
   const title = document.createElement("h2");
   title.textContent = product.title || "Untitled product";
   const brand = document.createElement("p");
   brand.className = "brand";
   brand.textContent = `Brand: ${product.brand || "Not available"}`;
   const price = document.createElement("p");
   price.className = "price";
   price.textContent = `$${Number(product.price).toFixed(2)}`;
   info.append(title, brand, price);
```

```
    card.append(image, info);
    productsGrid.appendChild(card);
  });
})
.catch((error) => {
  showError(error.message || "Could not load products. Check your internet connection.");
  console.error("Product fetch error:", error);
});
```

Output:




# Question 3 – Recipe Finder

Q3.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Recipe Finder</title>
 <link rel="stylesheet" href="Q3_recipes.css">
 <script src="Q3_recipes.js" defer></script>
</head>
<body>
 <header class="hero">
  <p class="eyebrow">WHAT'S COOKING?</p>
  <h1>Recipe Finder</h1>
  <p>Discover a meal and see what goes into it.</p>
 </header>
 <main class="container">
  <p id="statusMessage" class="status" role="status">Loading recipes...</p>
  <p id="httpStatus" class="small">HTTP status: waiting...</p>
  <section id="recipesGrid" class="grid" aria-label="Recipes"></section>
  <section id="recipeDetails" class="details" aria-live="polite">
   <p>Select a recipe to view its ingredients.</p>
  </section>
 </main>
</body>
</html>
```

Q3.css

```css
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, Helvetica, sans-serif; background: #fffaf4; color: #302a26; }
.hero { padding: 40px 16px; text-align: center; color: white; background: linear-gradient(135deg,
#8d351f, #d97736); }
.hero h1 { margin: 6px 0; font-size: clamp(2rem, 5vw, 3rem); }
.hero p { margin: 8px 0 0; }
.eyebrow { color: #ffe2c6; letter-spacing: 3px; font-weight: bold; font-size: .78rem; }
.container { width: min(1150px, 92%); margin: 25px auto 50px; }
.status { font-weight: bold; }
.small { color: #78716c; font-size: .9rem; }
.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(100%, 245px), 1fr)); gap:
18px; margin: 20px 0 28px; }
.recipe-card { background: white; border: 1px solid #f0dfd0; border-radius: 12px; overflow: hidden;
box-shadow: 0 4px 14px #6b34120d; }
.recipe-card img { width: 100%; height: 175px; object-fit: cover; background: #f3e8dc; }
.recipe-info { padding: 15px; }
.recipe-info h2 { margin: 0 0 9px; font-size: 1.1rem; }
.recipe-info p { margin: 5px 0; color: #78716c; font-size: .92rem; }
.recipe-info button { margin-top: 10px; padding: 9px 12px; background: #b45309; border: 0; border-
radius: 7px; color: white; cursor: pointer; font-weight: bold; }
.recipe-info button:hover { background: #92400e; }
.details { background: white; border: 1px solid #f0dfd0; border-radius: 12px; padding: 22px; }
.details h2 { margin-top: 0; }
.details li { margin: 7px 0; }
.error { color: #b91c1c; }
```

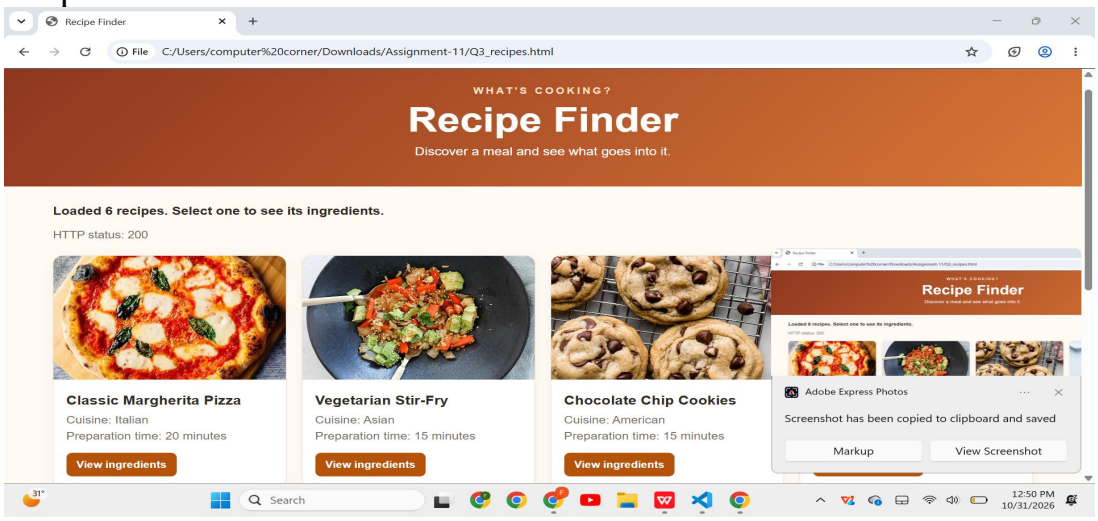
Q3.js

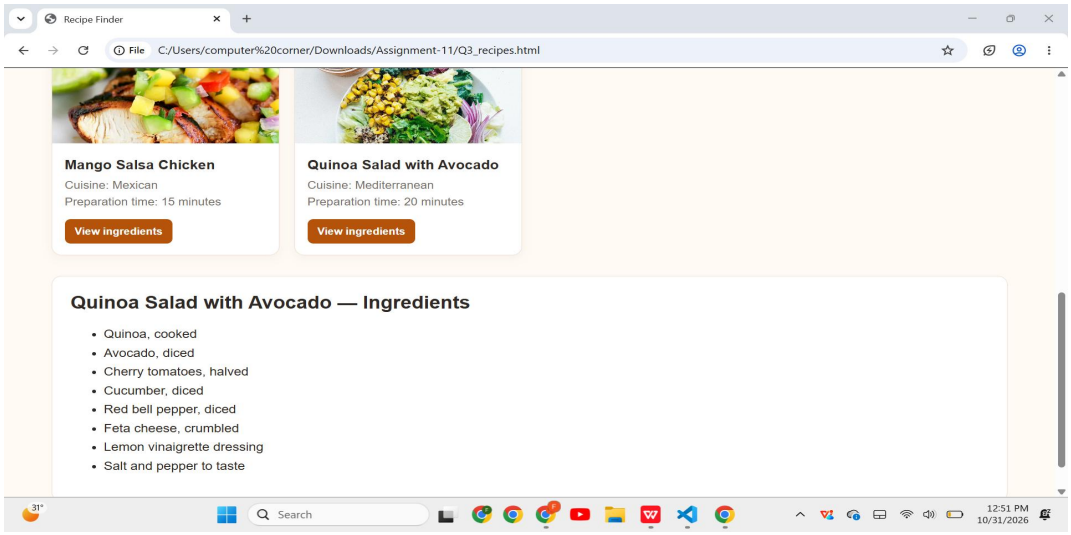
```js
const API_URL = "https://dummyjson.com/recipes";
const recipesGrid = document.getElementById("recipesGrid");
const statusMessage = document.getElementById("statusMessage");
const httpStatus = document.getElementById("httpStatus");
const recipeDetails = document.getElementById("recipeDetails");

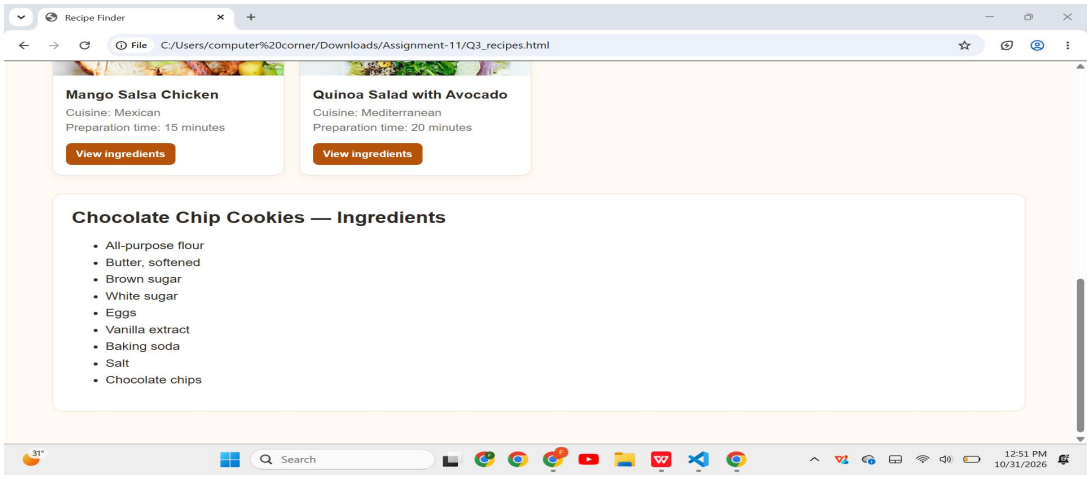
function showError(message) {
  statusMessage.textContent = message;
  statusMessage.className = "status error";
}
function showRecipeDetails(recipe) {
  recipeDetails.replaceChildren();
  const heading = document.createElement("h2");
  heading.textContent = `${recipe.name} — Ingredients`;
  const list = document.createElement("ul");
  (recipe.ingredients || []).forEach((ingredient) => {
    const item = document.createElement("li");
    item.textContent = ingredient;
    list.appendChild(item);
  });
  if (!recipe.ingredients || recipe.ingredients.length === 0) {
    const item = document.createElement("li");
    item.textContent = "Ingredients are not available for this recipe.";
    list.appendChild(item);
  }
  recipeDetails.append(heading, list);
}
async function loadRecipes() {
  statusMessage.textContent = "Loading recipes...";
```

```
 try {
   const response = await fetch(API_URL);
   httpStatus.textContent = `HTTP status: ${response.status} ${response.statusText}`;
   console.log("Recipe API status:", response.status);
   if (!response.ok) {
     throw new Error(`The recipe request failed (HTTP ${response.status}).`);
   }
   const data = await response.json();
   const recipes = data.recipes;
   if (!Array.isArray(recipes) || recipes.length === 0) {
     throw new Error("The API did not return any recipes.");
   }
   recipesGrid.replaceChildren();
   recipes.slice(0, 6).forEach((recipe) => {
     const card = document.createElement("article");
     card.className = "recipe-card";
     const image = document.createElement("img");
     image.src = recipe.image || "";
     image.alt = recipe.name || "Recipe image";
     image.loading = "lazy";
     if (!recipe.image) image.hidden = true;
     image.addEventListener("error", () => { image.hidden = true; });
     const info = document.createElement("div");
     info.className = "recipe-info";
     const title = document.createElement("h2");
     title.textContent = recipe.name || "Unnamed recipe";
     const cuisine = document.createElement("p");
     cuisine.textContent = `Cuisine: ${recipe.cuisine || "Not available"}`;
     const time = document.createElement("p");
     const prep = recipe.prepTimeMinutes;
     time.textContent = `Preparation time: ${prep !== undefined ? `${prep} minutes` : "Not available"}`;
     const button = document.createElement("button");
     button.type = "button";
     button.textContent = "View ingredients";
     button.addEventListener("click", () => showRecipeDetails(recipe));
     info.append(title, cuisine, time, button);
     card.append(image, info);
     recipesGrid.appendChild(card);
   });
   statusMessage.textContent = `Loaded ${Math.min(6, recipes.length)} recipes. Select one to see its
ingredients.`;
   statusMessage.className = "status";
 } catch (error) {
   showError(error.message || "Could not load recipes. Check your internet connection.");
   console.error("Recipe fetch error:", error);
 }
}
loadRecipes();
```

Output:




# Question 4 – API Request Practice: Create and Update a Post

Q4.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Post Request Practice</title>
  <link rel="stylesheet" href="Q4_postRequests.css">
  <script src="Q4_postRequests.js" defer></script>
</head>
<body>
  <header class="hero">
    <p class="eyebrow">JAVASCRIPT • FETCH API</p>
    <h1>Create and Update a Post</h1>
    <p>Practice POST, PUT, PATCH, and DELETE requests.</p>
  </header>
  <main class="container">
    <section class="panel">
      <h2>Create a new post</h2>
      <form id="postForm">
        <label for="title">Title</label>
        <input id="title" name="title" type="text" placeholder="Announcement title" required>
        <label for="content">Content</label>
        <textarea id="content" name="content" rows="4" placeholder="Write the announcement..."
required></textarea>
        <label for="userId">User ID</label>
        <input id="userId" name="userId" type="number" min="1" value="1" required>
        <button id="createButton" type="submit">Send POST request</button>
      </form>
    </section>

    <section class="panel">
      <h2>Update or delete post 1</h2>
      <p class="note">These buttons send requests to <code>/posts/1</code>. JSONPlaceholder
demonstrates the response but does not permanently change its data.</p>
      <div class="actions">
        <button id="putButton" type="button" class="orange">Send PUT</button>
        <button id="patchButton" type="button" class="green">Send PATCH</button>
        <button id="deleteButton" type="button" class="danger">Send DELETE</button>
      </div>
    </section>
    <section class="panel">
      <h2>Response</h2>
      <p id="message" role="status">Ready to send a request.</p>
      <p id="statusCode" class="status-code">HTTP status: —</p>
      <pre id="responseOutput">The API response will appear here.</pre>
    </section>
  </main>
</body>
</html>
```

## Q4.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, Helvetica, sans-serif; color: #202938; background: #f3f6fb; line-
height: 1.5; }
.hero { text-align: center; padding: 38px 16px; background: #183b67; color: white; }
.hero h1 { margin: 5px 0; font-size: clamp(1.8rem, 5vw, 2.7rem); }
.hero p { margin: 7px 0 0; color: #dbeafe; }
.eyebrow { font-size: .78rem; letter-spacing: 2px; font-weight: bold; }
.container { width: min(850px, 92%); margin: 26px auto 45px; }
.panel { background: white; padding: 22px; margin-bottom: 20px; border-radius: 12px; box-shadow: 0
4px 16px #172b4d10; }
```

```
h2 { margin-top: 0; }
label { display: block; font-weight: bold; margin: 12px 0 5px; }
input, textarea { width: 100%; border: 1px solid #cbd5e1; border-radius: 7px; padding: 11px; font:
inherit; }
textarea { resize: vertical; }
button { border: 0; border-radius: 7px; padding: 11px 15px; color: white; background: #2563eb; cursor:
pointer; font-weight: bold; margin-top: 14px; }
button:hover { filter: brightness(.93); }
button:disabled { opacity: .55; cursor: wait; }
.orange { background: #b45309; }
.green { background: #047857; }
.danger { background: #b91c1c; }
.actions { display: flex; flex-wrap: wrap; gap: 9px; }
.note { color: #64748b; }
#message { font-weight: bold; }
.status-code { color: #475569; }
pre { padding: 16px; border-radius: 8px; background: #0f172a; color: #dbeafe; white-space: pre-wrap;
overflow-wrap: anywhere; min-height: 90px; }
.error { color: #b91c1c; }
.success { color: #047857; }
@media (max-width: 520px) { .panel { padding: 16px; } .actions button { flex: 1 1 100%; } }
```

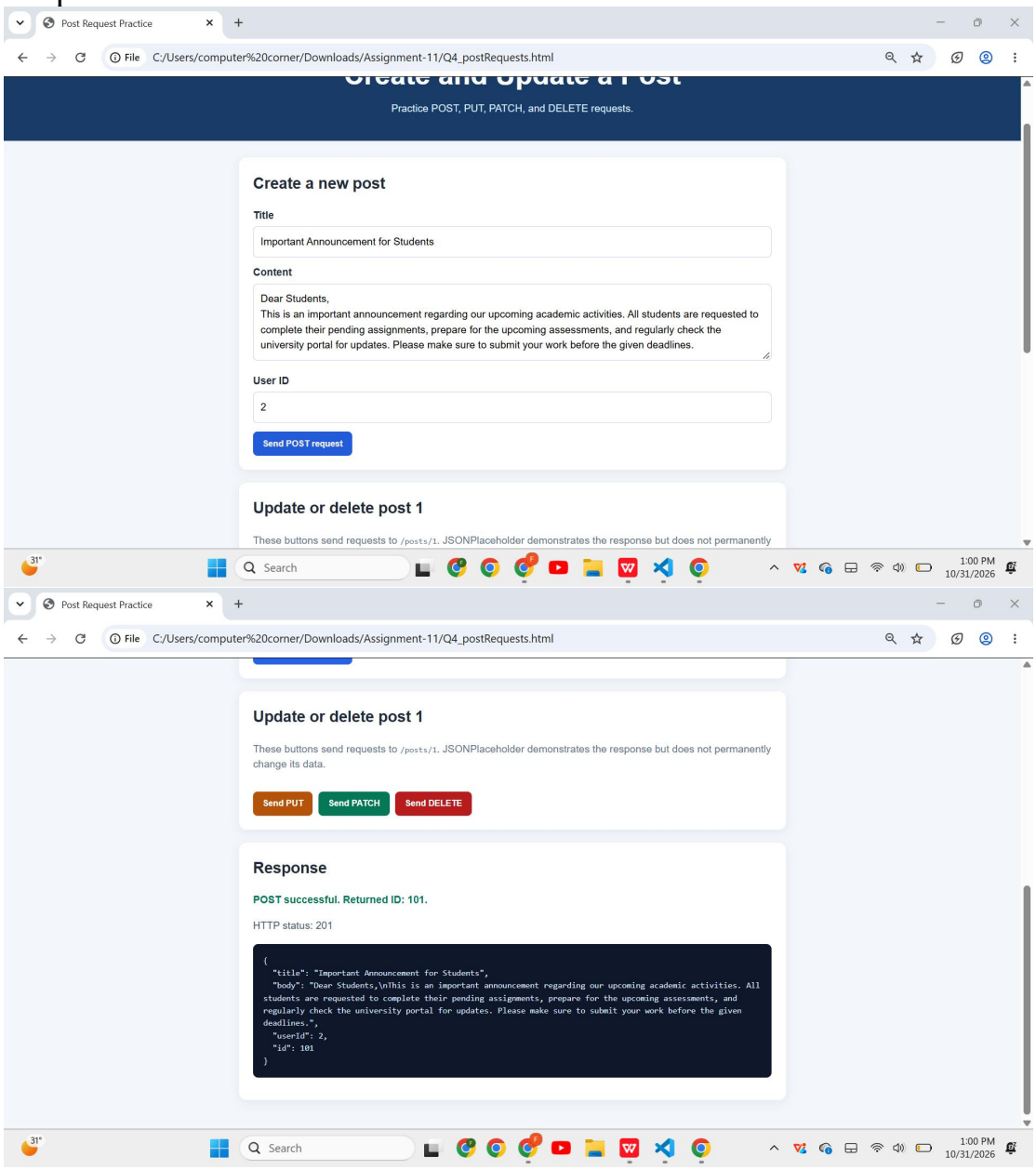
## Q4.js

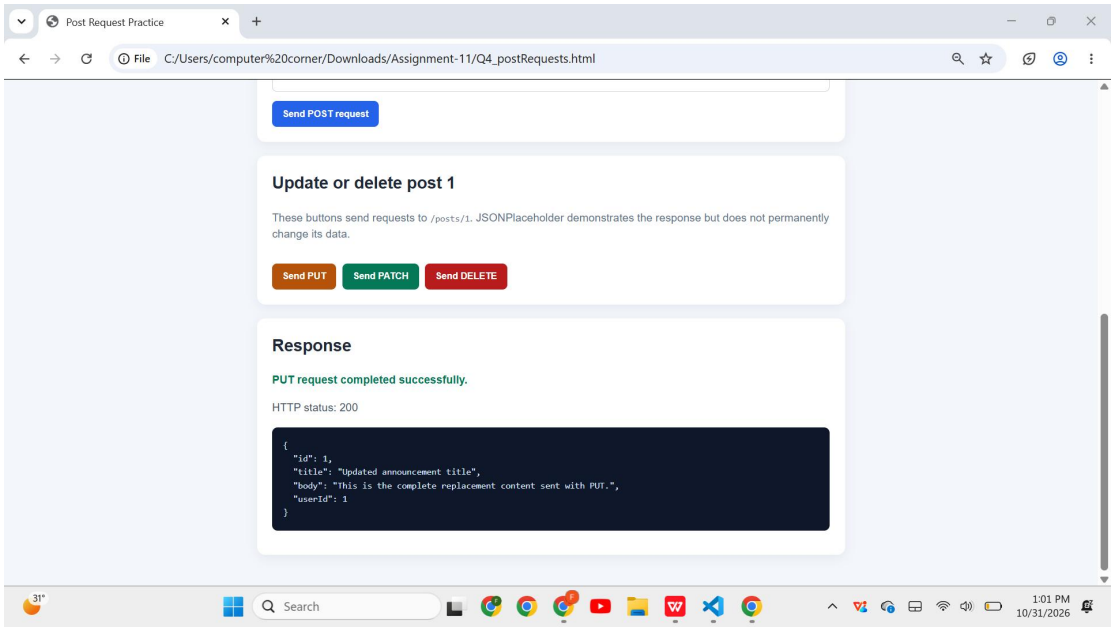
```
const API_URL = "https://jsonplaceholder.typicode.com/posts";
const postForm = document.getElementById("postForm");
const createButton = document.getElementById("createButton");
const putButton = document.getElementById("putButton");
const patchButton = document.getElementById("patchButton");
const deleteButton = document.getElementById("deleteButton");
const message = document.getElementById("message");
const statusCode = document.getElementById("statusCode");
const responseOutput = document.getElementById("responseOutput");

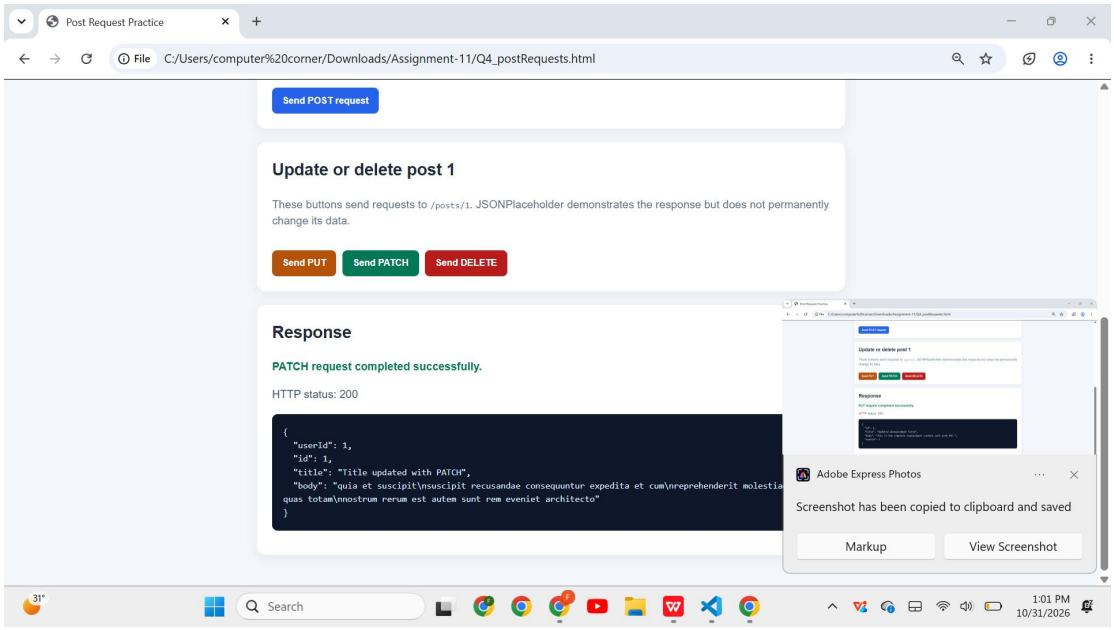
function showMessage(text, type = "") {
 message.textContent = text;
 message.className = type;
}
// Reusable helper: send a request with a URL, HTTP method, and optional body.
async function sendRequest(url, method = "GET", body = undefined) {
 const options = { method, headers: {} };
 if (body !== undefined) {
   // JSON.stringify converts a JavaScript object into JSON text for transmission.
   // Content-Type tells the server that the request body contains JSON.
   options.headers["Content-Type"] = "application/json; charset=UTF-8";
   options.body = JSON.stringify(body);
 }
 const response = await fetch(url, options);
 statusCode.textContent = `HTTP status: ${response.status} ${response.statusText}`;
 console.log(`${method} status:`, response.status);
 // A 404/500 response does not automatically reject fetch, so check response.ok.
 const responseText = await response.text();
 let responseData = null;
 if (responseText) {
   try {
     responseData = JSON.parse(responseText);
   } catch {
     responseData = responseText;
   }
```

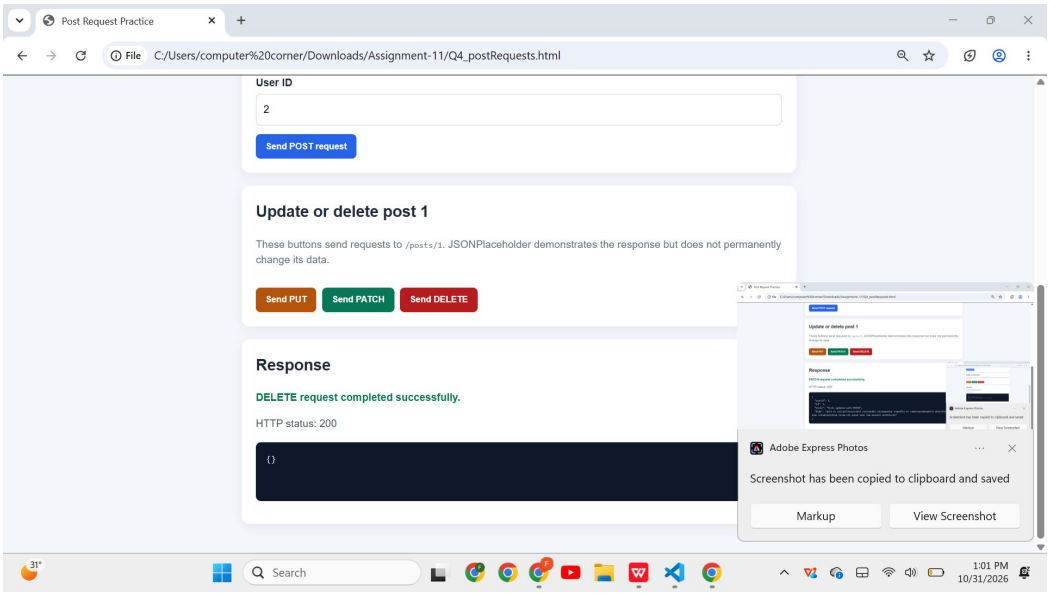
```
  }
  if (!response.ok) {
    const error = new Error(`${method} request failed (HTTP ${response.status}).`);
    error.status = response.status;
    error.responseData = responseData;
    throw error;
  }
  return { status: response.status, data: responseData };
}
async function runRequest(method, url, body) {
  const buttons = [createButton, putButton, patchButton, deleteButton];
  buttons.forEach((button) => { button.disabled = true; });
  showMessage(`Sending ${method} request...`);
  responseOutput.textContent = "Waiting for API response...";
  try {
    const result = await sendRequest(url, method, body);
    showMessage(`${method} request completed successfully.`, "success");
    responseOutput.textContent = JSON.stringify(result.data, null, 2) || "(No response body)";
    if (method === "POST" && result.data && result.data.id !== undefined) {
      showMessage(`POST successful. Returned ID: ${result.data.id}.`, "success");
    }
  } catch (error) {
    showMessage(error.message || "A network error occurred.", "error");
    responseOutput.textContent = error.responseData
      ? JSON.stringify(error.responseData, null, 2)
      : "No response data was available. Check your internet connection and try again.";
    console.error(`${method} request error:`, error);
  } finally {
    buttons.forEach((button) => { button.disabled = false; });
  }
}
postForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const title = document.getElementById("title").value.trim();
  const body = document.getElementById("content").value.trim();
  const userId = Number(document.getElementById("userId").value);
  if (!title || !body || !Number.isInteger(userId) || userId < 1) {
    showMessage("Please enter a title, content, and valid positive user ID.", "error");
    return;
  }
  runRequest("POST", API_URL, { title, body, userId });
});
putButton.addEventListener("click", () => {
  // PUT sends the complete replacement representation of the post.
  runRequest("PUT", `${API_URL}/1`, {
    id: 1,
    title: "Updated announcement title",
    body: "This is the complete replacement content sent with PUT.",
    userId: 1
  });
});
patchButton.addEventListener("click", () => {
  // PATCH sends only the field that should be changed.
  runRequest("PATCH", `${API_URL}/1`, {
    title: "Title updated with PATCH"
  });
});
deleteButton.addEventListener("click", () => {
  runRequest("DELETE", `${API_URL}/1`);
});
```

Output:


Post Request Practice
File C:/Users/computer%20corner/Downloads/Assignment-11/Q4_postRequests.html
Practice POST, PUT, PATCH, and DELETE requests.
Create a new post
Title
Important Announcement for Students
Content
Dear Students,
This is an important announcement regarding our upcoming academic activities. All students are requested to complete their pending assignments, prepare for the upcoming assessments, and regularly check the university portal for updates. Please make sure to submit your work before the given deadlines.
User ID
2
Send POST request
Update or delete post 1
These buttons send requests to /posts/1. JSONPlaceholder demonstrates the response but does not permanently
Search
1:00 PM
10/31/2026
Post Request Practice
File C:/Users/computer%20corner/Downloads/Assignment-11/Q4_postRequests.html
Update or delete post 1
These buttons send requests to /posts/1. JSONPlaceholder demonstrates the response but does not permanently change its data.
Send PUT
Send PATCH
Send DELETE
Response
POST successful. Returned ID: 101.
HTTP status: 201
{
"title": "Important Announcement for Students",
"body": "Dear Students,\nThis is an important announcement regarding our upcoming academic activities. All students are requested to complete their pending assignments, prepare for the upcoming assessments, and regularly check the university portal for updates. Please make sure to submit your work before the given deadlines.",
"userId": 2,
"id": 101
}
Search
1:00 PM
10/31/2026


Post Request Practice
File C:/Users/computer%20corner/Downloads/Assignment-11/Q4_postRequests.html
Send POST request
Update or delete post 1
These buttons send requests to /posts/1. JSONPlaceholder demonstrates the response but does not permanently change its data.
Send PUT
Send PATCH
Send DELETE
Response
PUT request completed successfully.
HTTP status: 200
{
"id": 1,
"title": "Updated announcement title",
"body": "This is the complete replacement content sent with PUT.",
"userId": 1
}
Search
1:01 PM
10/31/2026


Post Request Practice
File C:/Users/computer%20corner/Downloads/Assignment-11/Q4_postRequests.html
Send POST request
Update or delete post 1
These buttons send requests to /posts/1. JSONPlaceholder demonstrates the response but does not permanently change its data.
Send PUT
Send PATCH
Send DELETE
Response
PATCH request completed successfully.
HTTP status: 200
{
"userId": 1,
"id": 1,
"title": "Title updated with PATCH",
"body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestia
quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
Adobe Express Photos
Screenshot has been copied to clipboard and saved
Markup
View Screenshot
Search
1:01 PM
10/31/2026


Post Request Practice
File C:/Users/computer%20corner/Downloads/Assignment-11/Q4_postRequests.html
User ID
2
Send POST request
Update or delete post 1
These buttons send requests to /posts/1. JSONPlaceholder demonstrates the response but does not permanently change its data.
Send PUT
Send PATCH
Send DELETE
Response
DELETE request completed successfully.
HTTP status: 200
{}
Adobe Express Photos
Screenshot has been copied to clipboard and saved
Markup
View Screenshot
Search
1:01 PM
10/31/2026